



SRI LANKA INSTITUTE OF ADVANCED TECHNOLOGICAL EDUCATION
Higher National Diploma in Information Technology

HNDIT 4052
Programming Individual Project

SOFTWARE REQUIREMENT SPECIFICATION
Hall Booking and Management System

Supervisor:
Ms.P.G.R.N.J.Gamlath

S.F Safiya
KUR/IT/2324/F/0080

ACADEMIC YEAR: 2324
YEAR II, SEMESTER II

CONTENTS

1.	INTRODUCTION	4
1.1	PURPOSE	4
1.2	SCOPE OF THE SYSTEM.....	4
1.3	DEFINITIONS AND ACRONYMS	5
1.4	SYSTEM OVERVIEW	6
2.	GENERAL DESCRIPTION.....	7
2.1	PRODUCT PERSPECTIVE	7
2.2	PRODUCT FUNCTIONS	7
2.3	USER CLASSES AND CHARACTERISTICS	8
2.4	FEATURES AND BENEFITS	9
2.5	ASSUMPTIONS AND DEPENDENCIES	10
3.	FUNCTIONAL REQUIREMENTS	11
3.1	USER AUTHENTICATION	11
3.2	HALL MANAGEMENT	11
3.3	BOOKING MANAGEMENT	12
3.4	PACKAGE AND SERVICE MANAGEMENT.....	12
3.5	PAYMENT MANAGEMENT	13
3.6	NOTIFICATION SYSTEM.....	13
3.7	REPORTING AND ANALYTICS	14
3.8	USER ROLE MANAGEMENT.....	14
4.	INTERFACE REQUIREMENTS.....	16
4.1	USER INTERFACE	16
4.2	SOFTWARE INTERFACE.....	16
4.3	COMMUNICATION INTERFACE.....	17
5.	DESIGN CONSTRAINTS	18
5.1	TECHNOLOGY STACK.....	18

5.2	CLOUD-BASED ARCHITECTURE CONSTRAINT	18
5.3	BOOKING VALIDATION CONSTRAINT	19
5.4	SECURITY CONSTRAINT	19
5.5	RESPONSIVE DESIGN CONSTRAINT	20
5.6	BROWSER COMPATIBILITY	20
5.7	SINGLE ORGANIZATION CONSTRAINT	21
5.8	VERSION CONTROL CONSTRAINT	21
6.	NON-FUNCTIONAL REQUIREMENTS	23
6.1	PERFORMANCE	23
6.2	SECURITY	23
6.3	RELIABILITY	24
6.4	USABILITY	24
6.5	SCALABILITY	25
6.6	MAINTAINABILITY	25
7.	APPENDICES	26
7.1	ASSUMPTIONS	26
7.2	DEPENDENCIES	26
7.3	ACRONYMS	26
7.4	USE CASE DIAGRAM	27
7.5	ENTITY RELATIONSHIP DIAGRAM	28
7.6	REFERENCES	30

1. INTRODUCTION

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) document is to comprehensively define and describe all the functional and non-functional requirements of the HallCamilla Booking and Management System developed for Camilla Banquet Hotel. This document serves as a formal reference between the software developer, the academic supervisor, and the project stakeholders by clearly describing how the application is expected to operate under various functional and operational scenarios.

Furthermore, the SRS ensures that all technical requirements, business rules, system constraints, and user interface behaviours are properly documented and understood before implementation and final deployment. By clearly specifying the expected functionality of the system, it helps minimise misunderstandings among stakeholders and provides a structured foundation for the successful development of the application.

In addition, this document serves as a continuous reference throughout the Software Development Life Cycle (SDLC). It provides clear guidance during the system analysis, design, implementation, testing, and evaluation phases, ensuring that every stage of development aligns with the defined requirements. As a result, the final HallCamilla Booking and Management System satisfies both the academic objectives of the project and the practical operational needs of Camilla Banquet Hotel.

1.2 Scope of the System

The HallCamilla Booking and Management System is a modern, serverless, cloud-based web application developed to replace the traditional manual booking methods used by Camilla Banquet Hotel. Conventional hall reservation processes often rely on physical record books, telephone inquiries, and paper-based payment records, making booking management time-consuming and increasing the likelihood of operational errors such as double bookings, inaccurate payment records, and inefficient customer communication.

The system provides a centralized digital platform that enables customers to browse available halls, view package details and pricing, submit booking requests, and complete online advance

payments through a secure payment gateway. Customers can also monitor the progress of their bookings, payment status, and notifications through a dedicated dashboard, improving convenience and transparency throughout the booking process.

In addition, HallCamilla provides administrators with a comprehensive management dashboard that supports real-time booking management, hall and package administration, payment tracking, customer management, and automated email notifications. By digitizing the complete booking workflow, the system improves operational efficiency, enhances data accuracy, reduces administrative workload, and provides a more convenient and professional booking experience for both customers and hotel administrators.

1.3 Definitions and Acronyms

This section explains the key technical terms and abbreviations used throughout this Software Requirements Specification (SRS) document to ensure clarity and consistency for all readers. Understanding these terms helps readers interpret the system requirements, architecture, and implementation details more effectively.

SRS stands for Software Requirements Specification, which defines the functional and non-functional requirements of the HallCamilla Booking and Management System. UI refers to the User Interface, representing the visual components and interactive screens through which users interact with the application. DB stands for Database, referring to the cloud-based data storage used to manage all application information. CRUD represents the four fundamental database operations: Create, Read, Update, and Delete, which are performed when managing system data.

API stands for Application Programming Interface, which enables secure communication between the HallCamilla system and external services such as PayHere and EmailJS. SPA refers to a Single Page Application, a web application architecture in which content is dynamically updated without requiring full page reloads, providing a faster and more responsive user experience. RBAC stands for Role-Based Access Control, a security mechanism that restricts access to system features based on the authenticated user's assigned role, such as Customer or Administrator. Finally, Firebase refers to Google's cloud-based Backend-as-a-Service (BaaS) platform, which provides authentication, Cloud Firestore database services, hosting, and other backend functionality used throughout the HallCamilla system.

1.4 System Overview

The HallCamilla Booking and Management System is developed as a modern, responsive, cloud-based web application using React.js, Vite, and Firebase cloud services. The system follows a serverless architecture, where backend functionalities such as user authentication, database management, and data synchronization are provided through Firebase services rather than a custom-built application server. This architectural approach simplifies deployment, reduces infrastructure management requirements, and enables scalable, real-time application performance.

The application is designed to support two primary user roles: Administrator and Customer. Administrators are provided with privileged access to manage hall information, service packages, booking requests, payment records, customer information, and other administrative operations through a dedicated management dashboard. Customers, on the other hand, can register accounts, browse available halls and packages, check hall availability, submit booking requests, make online payments, track booking progress, receive notifications, and manage their personal profiles through the customer dashboard.

To ensure system security and efficient data management, role-based access control is implemented using Firebase Authentication and Cloud Firestore. This mechanism restricts users to features appropriate to their assigned roles while protecting sensitive administrative functions. In addition, the system utilizes Firestore's real-time synchronization capabilities, allowing updates such as booking approvals, payment status changes, and notifications to be reflected immediately across the application without requiring users to manually refresh their web browsers. This real-time functionality enhances both operational efficiency and the overall user experience.

2. GENERAL DESCRIPTION

2.1 Product Perspective

The HallCamilla Booking and Management System is a comprehensive, standalone web application developed to centralize and streamline banquet hall booking and management operations. The system replaces traditional manual record-keeping methods with a fully digital workflow that improves operational efficiency, enhances data accuracy, and minimizes the risk of human error throughout the booking and payment processes.

From a technical perspective, the application follows a cloud-based, serverless architecture. Persistent application data is securely stored in Firebase Cloud Firestore, while user authentication and access control are managed through Firebase Authentication. This approach eliminates the need for a dedicated backend server while providing reliable, scalable, and secure cloud services for managing bookings, payments, notifications, and user information.

The presentation layer is developed using React.js together with Tailwind CSS to provide a responsive and user-friendly interface. The frontend communicates directly with Firebase services and integrates with external third-party platforms such as PayHere for online payment processing and EmailJS for automated email notifications. This modern serverless architecture makes HallCamilla a practical and cost-effective solution for small and medium-sized banquet halls seeking to digitize their booking operations without the complexity and maintenance costs associated with traditional server-based systems.

2.2 Product Functions

The HallCamilla system provides a comprehensive set of core functionalities designed to support efficient banquet hall operations and improve the overall booking experience for both customers and administrators.

The User Authentication module securely manages user registration and login processes for both customers and administrators. By integrating Firebase Authentication with role-based access control, the system ensures that users can only access features and information appropriate to their assigned roles.

The Hall Management module allows administrators to manage information related to the available banquet halls, including hall names, descriptions, pricing details, guest capacity, locations, and images. This enables hotel staff to maintain accurate and up-to-date venue information from a centralized dashboard.

The Booking Management module functions as the core operational component of the application. Customers can submit reservation requests by selecting their preferred hall, event date, package, and required details. Before creating a booking, the system performs availability validation using Firestore data queries to identify existing active reservations and prevent conflicting bookings for the same date.

The Package and Service Management module enables administrators to create and maintain different event packages, such as the Bronze Essential, Silver Celebration, and Garden Elegance packages. These packages allow customers to select suitable service options while ensuring that pricing information remains accurate and centrally managed.

The Payment Management module integrates with the PayHere Sandbox payment gateway to support secure online payment processing. The system automatically calculates the required twenty percent advance payment and eighty percent remaining balance based on the total booking amount. All payment transactions are recorded within the system, allowing administrators to monitor payment progress and maintain accurate financial records.

Finally, the Notification System improves communication between customers and administrators through automated email notifications and in-app alerts. The system generates notifications for important events such as booking approvals, payment confirmations, and status changes, ensuring that all users receive timely updates throughout the reservation lifecycle.

2.3 User Classes and Characteristics

The HallCamilla system is accessed by two primary user categories, each with clearly defined responsibilities, permissions, and system privileges.

The Administrator user class represents the hotel management team and authorized staff members who have elevated access rights within the system. Administrators are responsible for managing overall system operations, including configuring hall details, maintaining package information,

reviewing customer booking requests, approving or rejecting reservations based on business requirements, monitoring payment records, and managing operational information through the administrator dashboard. Administrators are expected to have basic computer literacy and an understanding of hotel booking workflows, enabling them to effectively use the management features provided by the system.

The Customer user class represents individuals who intend to reserve banquet halls for personal or organizational events. Customers interact with the system to browse available halls, view package details, check event date availability, submit booking requests, complete required payments, view booking progress, and receive notifications regarding their reservations. The customer interface is designed to be simple, user-friendly, and accessible without requiring advanced technical knowledge. To maintain privacy and security, customers have restricted access and can only view and manage information related to their own accounts, bookings, payments, and notifications.

2.4 Features and Benefits

Implementing the HallCamilla system provides significant improvements compared with traditional manual booking and record-keeping methods. The system reduces the risk of double bookings by performing real-time availability validation during the reservation process, ensuring that multiple customers cannot reserve the same hall for the same event date.

By centralizing booking information, customer details, payment records, and notification data within a secure cloud-based platform, the system improves data reliability and accessibility while reducing the risks associated with physical record loss or damage. Authorized users can access required information efficiently through the web application without depending on manually maintained documents.

The system also enhances the customer experience by allowing users to independently browse available halls, view package details, estimate booking costs, submit reservation requests, and complete payments through an online platform. This reduces the need for repeated phone inquiries or physical visits to the hotel and provides customers with greater convenience and transparency throughout the booking process.

From an administrative perspective, HallCamilla reduces routine manual workload through automated booking management, payment tracking, real-time updates, and notification services. These improvements allow hotel staff to manage reservations more efficiently and focus greater attention on event coordination and customer service activities.

2.5 Assumptions and Dependencies

The successful operation of the HallCamilla system depends on several important assumptions and technical dependencies. Since the application follows a cloud-based architecture, it is assumed that both customers and administrators have access to a stable internet connection through broadband or mobile networks. Continuous connectivity is required because the system relies on online services for authentication, data retrieval, booking management, payment processing, and real-time updates.

The system is dependent on the availability and reliability of Google Firebase services, particularly Firebase Authentication for managing user identity verification and access control, and Cloud Firestore for storing, retrieving, and synchronizing application data. Any interruption in these cloud services may temporarily affect system functionality, including login operations and access to booking information.

In addition, HallCamilla depends on external third-party services such as the PayHere Sandbox API for handling online payment transactions and EmailJS for delivering automated email notifications. The availability and proper functioning of these services are required to maintain smooth payment processing and communication workflows.

Finally, it is assumed that hotel administrators will regularly maintain and update system information, including hall details, pricing information, and package configurations. Accurate administrative updates are necessary to ensure that customers always receive correct and up-to-date information when making booking decisions.

3. FUNCTIONAL REQUIREMENTS

3.1 User Authentication

The system must implement a secure authentication mechanism using Firebase Authentication services to manage user identity verification and access control. All users are required to create an account using a valid email address and password before they can access protected system features or submit booking requests. During registration and login processes, Firebase Authentication securely manages user credentials and maintains authenticated user sessions.

The application must also implement strict Role-Based Access Control (RBAC) by retrieving the user's assigned role from the Firestore users collection after successful authentication. Based on this role information, the system determines the level of access granted to each user. This ensures that administrative dashboard features and management operations are accessible only to authorized administrator accounts, while customer accounts remain restricted to customer-specific functionalities.

By enforcing authentication and role-based authorization, the system protects sensitive business data, prevents unauthorized access to administrative functions, and ensures that users can only perform operations appropriate to their assigned permissions.

3.2 Hall Management

The Hall Management module must be accessible exclusively to authenticated administrator users, ensuring that only authorized personnel can modify venue-related information. This module must provide administrators with complete management capabilities, including the ability to create new hall profiles, update existing hall details, and remove halls from the active system database when they are no longer available for reservations.

Each hall record must contain essential operational information required for effective booking management, including the hall name, maximum guest capacity, base price, descriptive details, and relevant images for customer display. The system must validate and maintain this information to ensure that accurate hall details are available throughout the application.

Any changes made by administrators within the Hall Management module must be synchronized with the customer-facing interface through the real-time database functionality provided by Cloud Firestore. This ensures that customers always view the latest available information regarding hall availability, pricing, capacity, and other venue details when making booking decisions.

3.3 Booking Management

The Booking Management module must function as the central operational component of the HallCamilla system. The system must allow authenticated customers to submit reservation requests by selecting a preferred hall, event package, expected guest count, and desired event date.

Before creating a new booking record, the system must perform a real-time availability validation process using Cloud Firestore to check whether the selected hall already has an active reservation for the requested event date. If an existing approved or confirmed booking is detected for the same hall and date, the system must prevent the new reservation request from being submitted and display an appropriate notification to the customer.

All successfully submitted booking requests must initially be assigned a "Pending" status. These requests must then become available for review through the administrator dashboard, where only authorized administrators can evaluate the reservation details and update the booking status to either "Approved" or "Rejected" according to the hotel's operational decisions and business requirements.

3.4 Package and Service Management

The Package Management module must support the creation and maintenance of structured event packages to provide customers with clear and consistent pricing options. The system must allow authenticated administrators to create, update, and manage different event package tiers, such as the Bronze Essential, Silver Celebration, and Garden Elegance packages.

Administrators must be able to configure important package attributes, including the package name, package price, included services, guest capacity limitations, and any additional pricing rules associated with exceeding the included guest limit. Maintaining these package details within a centralized management module ensures that pricing information remains accurate and consistent throughout the system.

When customers proceed with the booking process, the system must automatically calculate the estimated total event cost based on the selected hall's base price and the chosen package details. This dynamic calculation ensures that customers receive an accurate cost breakdown before submitting their reservation request, improving pricing transparency and reducing the possibility of manual calculation errors.

3.5 Payment Management

The Payment Management module must handle and monitor all financial transactions within the HallCamilla system through integration with the PayHere Sandbox payment gateway. The system must enforce the hotel's defined payment workflow by requiring a 20% advance payment after a booking receives administrative approval and collecting the remaining 80% balance payment closer to the scheduled event date.

The system must securely generate the required payment information and transmit the payment payload to the PayHere interface for online transaction processing. After receiving a successful payment response from the gateway, the system must create a corresponding payment record within the dedicated Payment collection and update the related booking document with the latest payment status to maintain synchronization between booking and financial records.

In addition to online payments, the system must provide administrators with the ability to manually record offline cash payments through the management dashboard. This functionality ensures that both online and offline transactions are accurately maintained within the system, providing administrators with a complete and reliable view of the hotel's financial records.

3.6 Notification System

The Notification Management module must provide an automated communication mechanism between the HallCamilla system, administrators, and customers. The system must automatically generate and deliver notifications whenever important operational events occur, including new booking submissions, administrative approval or rejection of requests, and successful payment confirmations.

To support external communication, the system must integrate with the EmailJS service to send formatted email notifications directly to the relevant users. These automated emails ensure that

customers and administrators receive timely updates regarding important changes in the booking and payment lifecycle without requiring manual communication from hotel staff.

In addition to email notifications, the system must maintain an internal notification system that displays alerts within the user's dashboard notification panel. These in-app notifications provide users with immediate access to important updates while improving communication efficiency and ensuring that all parties remain informed throughout the reservation process.

3.7 Reporting and Analytics

The Reporting and Analytics module must provide administrators with meaningful business insights through the management dashboard. The system must process and summarize information stored within the database to generate clear performance indicators related to booking activities and financial operations.

The module must display important analytical information, including total revenue collected, the number of pending and completed bookings, payment status summaries, and recent transaction records. These insights allow administrators to monitor reservation performance, track outstanding payments, and evaluate the overall operational status of the hotel.

The reporting information must be updated dynamically based on changes in the system data, ensuring that administrators have access to accurate and current business information. By providing real-time analytical views, the module supports faster decision-making and helps management effectively monitor the performance and financial health of Camilla Banquet Hotel.

3.8 User Role Management

The system must enforce user permissions through a structured role management architecture to ensure that each user can only access features appropriate to their assigned role. The application must distinguish between Customer and Administrator roles by retrieving the user's assigned access level from the securely maintained user records after authentication.

Administrators must be provided with authorized access to all operational, financial, and system management functionalities, including booking management, hall configuration, package management, payment monitoring, and reporting features. In contrast, customers must have

restricted access limited to customer-specific functions, such as browsing available halls, submitting bookings, viewing their own reservation history, managing payments, and receiving notifications.

To maintain system security, the application must implement protected routing mechanisms within the React frontend to prevent unauthorized users from accessing restricted pages. Additionally, Firestore security rules must be configured to control database-level access and ensure that users cannot retrieve or modify information beyond their permitted scope. These security measures work together to prevent unauthorized access and maintain the confidentiality and integrity of system data.

4. INTERFACE REQUIREMENTS

4.1 User Interface

The application must provide a modern, professional, and user-friendly interface developed using the Tailwind CSS framework. The user interface must follow responsive web design principles to ensure that all pages, booking forms, dashboards, and system components adapt effectively across different screen sizes, including desktop computers, tablets, and mobile devices.

The system must provide clearly separated user interfaces for customers and administrators, allowing each user category to access features relevant to their responsibilities without causing navigation confusion. The customer interface must focus on simplicity and ease of use, enabling users to browse halls, select packages, submit bookings, complete payments, and monitor reservation updates efficiently.

The administrator interface must provide an organized dashboard environment that allows staff members to manage bookings, halls, packages, payments, notifications, and reports effectively. Overall, the interface design must prioritize usability, accessibility, and intuitive navigation, ensuring that users with different levels of technical experience can interact with the system successfully and complete required tasks with minimal difficulty.

4.2 Software Interface

The system must maintain reliable software interfaces to ensure seamless communication between the HallCamilla application, external services, and cloud-based infrastructure. The React.js frontend must integrate securely with the Google Firebase SDK to support user authentication processes and perform database operations through Cloud Firestore.

The application must also integrate with the PayHere Sandbox API to support online payment processing. The system must generate and transmit correctly formatted transaction data to initialize the payment gateway interface and must properly handle asynchronous responses returned after the completion of payment transactions. These responses must be processed to update payment records and maintain accurate booking payment statuses.

Additionally, the system must communicate with the EmailJS API to support automated email notifications. The application must securely provide the required email templates and dynamic data parameters to generate and deliver system-generated communications related to booking updates, payment confirmations, and other important events.

These external integrations must operate reliably to ensure smooth data exchange, accurate transaction processing, and effective communication throughout the HallCamilla system.

4.3 Communication Interface

All communication between the user's web browser and backend cloud services must be performed through secure HTTPS protocols to ensure that sensitive information, including customer details and payment-related data, is protected during transmission. This secure communication mechanism helps maintain data confidentiality and reduces the risk of unauthorized access or interception.

The system must utilize lightweight JSON-based data exchange formats for communication between the frontend application, cloud services, and external APIs. This approach supports efficient data transfer, improves application performance, and enables smooth interaction between different system components.

In addition to standard data request and response operations, the system must utilize real-time communication capabilities provided by the Firebase SDK through persistent WebSocket connections. These connections allow Cloud Firestore to automatically deliver live database updates to the React application whenever relevant data changes occur. This functionality enables features such as real-time booking status updates, payment status changes, and notification delivery without requiring users to manually refresh the application.

5. DESIGN CONSTRAINTS

5.1 Technology Stack

The development of the HallCamilla system must follow a defined modern web technology stack to ensure performance, maintainability, scalability, and compliance with the project requirements. The user interface and client-side application logic must be developed using React.js together with the Vite build tool, providing efficient development, fast application loading, and an interactive user experience.

The system must utilize Firebase Cloud Firestore as the primary NoSQL database for storing and managing all persistent application data, including user information, hall details, bookings, payments, notifications, and reviews. Firebase Authentication must be used to securely manage user registration, login processes, and identity verification throughout the application.

The completed application must be deployed using Firebase Hosting to provide reliable, secure, and accessible delivery of the web application to end users. This technology stack ensures that the HallCamilla system can operate efficiently while maintaining a scalable cloud-based architecture suitable for modern web application requirements.

5.2 Cloud-Based Architecture Constraint

The HallCamilla system is fundamentally constrained by its serverless, cloud-based architectural model. Since the application relies on Backend-as-a-Service (BaaS) infrastructure instead of a locally hosted server environment, a continuous and reliable internet connection is required for proper operation. All essential functionalities, including user authentication, database access, real-time synchronization, and payment processing, depend on communication with cloud-based services.

Due to this dependency, the system cannot operate fully in an offline environment. However, this cloud-based approach provides several advantages, including improved availability, automatic scalability, centralized data management, and enhanced security features offered through Google Cloud infrastructure and Firebase services.

Therefore, while internet dependency is a significant operational constraint, the selected architecture ensures that the HallCamilla system can provide a reliable, scalable, and secure platform for managing banquet hall reservations and related business operations.

5.3 Booking Validation Constraint

The HallCamilla system is constrained by a full-day reservation model based on the operational requirements of the banquet hotel. The system must prevent multiple customers from reserving the same physical hall for the same calendar date, ensuring that each hall can only accommodate one confirmed event per day.

To enforce this business rule, the booking management logic must perform an availability validation process before creating a new reservation. The system checks Cloud Firestore for existing bookings with matching hall identifiers and event dates while considering active booking statuses such as Approved and Confirmed.

If a conflicting reservation is detected, the system must immediately prevent the new booking request from being submitted and inform the customer that the selected hall is unavailable for the requested date. This constraint ensures reliable reservation management and eliminates the risk of double-booking, protecting both customer satisfaction and hotel operations.

5.4 Security Constraint

The HallCamilla system must follow strict security constraints to protect sensitive user information, financial data, and business-related records. All administrative routes, database write operations, and restricted system functions must be protected through authentication verification mechanisms and Firestore Security Rules, ensuring that only authorized users with appropriate permissions can modify sensitive information.

The system must prevent the exposure of confidential credentials and sensitive configuration values within the client-side application. Important security information, such as payment gateway credentials and private API keys, must not be directly accessible through browser-based code. Proper security practices must be followed to ensure that external service integrations remain protected.

User password management must be handled securely through Firebase Authentication services. The system must rely on Firebase's encrypted authentication infrastructure for storing and validating user credentials, ensuring that neither developers nor hotel administrators have access to users' raw passwords. These security measures help maintain user privacy, protect business data, and reduce the risk of unauthorized access.

5.5 Responsive Design Constraint

The visual design of the HallCamilla system must follow responsive web design principles to ensure consistent usability across different devices and screen sizes. Since customers and administrators may access the application through various devices, including desktop computers, tablets, and mobile phones, all interface components must adapt dynamically according to the user's screen resolution.

The system must ensure that complex elements, such as administrator dashboard tables, booking forms, and customer interfaces, can resize, reorganize, and display information effectively on different screen dimensions. The application must utilize responsive layout techniques and Tailwind CSS utility classes to create flexible and adaptive user interfaces.

By implementing responsive design practices, HallCamilla ensures that users receive a consistent and professional experience regardless of the device being used. This approach improves accessibility, maintains interface quality, and allows customers and administrators to efficiently perform their required tasks from any supported platform.

5.6 Browser Compatibility

The HallCamilla application must be compatible with the latest stable versions of major modern web browsers to ensure reliable accessibility and consistent performance for all users. The supported browsers include Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari, which provide the necessary support for modern web technologies used within the application.

The system must ensure that the React.js framework, modern JavaScript features, and other frontend technologies implemented within the application function correctly across these supported browser environments. This compatibility requirement helps maintain a consistent user

experience, proper interface rendering, and reliable system functionality regardless of the user's preferred browser.

The application is not required to support outdated or deprecated browsers, such as Internet Explorer, because maintaining compatibility with obsolete technologies may negatively affect application performance, security, and the ability to utilize modern web development practices.

5.7 Single Organization Constraint

The database architecture and administrative functionality of the HallCamilla system are specifically designed to support a single business entity, namely Camilla Banquet Hotel located in Kurunegala. The application currently operates as a standalone solution developed to manage the reservation and operational requirements of this specific banquet venue.

The system maintains all hall information, booking records, user accounts, payment details, notifications, and administrative data within a single centralized environment. These records are managed according to the operational rules and policies defined by the hotel, ensuring consistent handling of reservations and related activities.

Although the current implementation focuses on a single hotel environment, future enhancements could extend the architecture to support multi-tenant or multi-branch operations. However, such functionality is outside the scope of the current system version and would require additional architectural modifications to support independent hotels, branches, and separate administrative structures.

5.8 Version Control Constraint

The HallCamilla software development process must follow professional version control practices to ensure effective management of source code changes throughout the entire development lifecycle. The developer must utilize the Git version control system to track modifications, manage feature implementations, and maintain a detailed history of code changes and bug fixes.

The central source code repository must be securely maintained using GitHub as the remote version control platform. This approach provides a reliable backup mechanism, reduces the risk of

data loss due to local hardware failures, and enables better organization and management of the project development process.

By adopting Git and GitHub, the project demonstrates alignment with modern software engineering practices, improves code maintainability, and provides a structured approach for monitoring the evolution of the HallCamilla system from initial development through final implementation.

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Performance

The HallCamilla system must be optimized to provide high performance, fast response times, and a smooth user experience. The application must load essential user interface components and retrieve required initial database information within a maximum period of three seconds under normal network conditions.

The use of the Vite build tool, optimized React components, and efficient frontend development practices must ensure that navigation between different sections of the Single Page Application (SPA) occurs quickly without unnecessary page reloads. This approach improves application responsiveness and provides users with a seamless interaction experience.

Furthermore, the system must utilize Firebase Cloud Firestore real-time listeners to deliver immediate data synchronization between the database and client application. Any important updates, such as booking approvals, payment status changes, or notification updates, must be reflected on the user interface with minimal delay, ensuring that administrators and customers always have access to the most current system information.

6.2 Security

System security is a critical non-functional requirement of the HallCamilla application. The system must ensure that all data communication between the client application, cloud services, and external integrations is protected through industry-standard HTTPS protocols using TLS/SSL encryption. This ensures that sensitive information, including user details, booking records, and payment-related data, remains protected during transmission.

The authentication module must utilize the secure identity management features provided by Firebase Authentication to protect user credentials and manage authenticated sessions. User passwords must be securely processed and stored by Firebase without exposing raw password information to developers or system administrators.

Furthermore, the Cloud Firestore database must be protected using properly configured Firestore Security Rules. These rules must validate the authentication status and assigned user role before

allowing any read or write operation. By enforcing role-based access restrictions at the database level, the system prevents unauthorized access, modification, or deletion of sensitive customer and business data, ensuring confidentiality, integrity, and overall system security.

6.3 Reliability

The HallCamilla system must demonstrate a high level of operational reliability to ensure continuous availability and minimize disruptions to hotel booking activities. By utilizing Google Firebase cloud infrastructure, the application benefits from a reliable and scalable hosting environment that reduces the risks associated with traditional locally hosted server systems.

The system must be designed to maintain high availability and provide dependable access to essential features, including user authentication, booking management, payment tracking, and notification services. The cloud-based architecture helps minimize service interruptions and supports stable operation during normal business activities.

Cloud Firestore must provide reliable data storage and synchronization mechanisms to protect important application records, including customer information, booking details, and payment transactions. These capabilities help reduce the risk of data loss caused by unexpected failures and ensure that critical business information remains securely maintained and accessible when required.

6.4 Usability

The HallCamilla system must be designed with a strong focus on usability and user experience to ensure that both customers and administrators can interact with the application efficiently. The user interface must be clean, well-organized, and visually consistent, following familiar design patterns that reduce the learning effort required to understand and operate the system.

The application must provide clear, meaningful, and user-friendly feedback whenever users perform invalid actions or encounter issues. For example, when a customer attempts to reserve a hall that is already unavailable, the system must display an understandable notification explaining the situation instead of presenting technical error messages or system codes.

The overall user experience must allow customers to complete the booking process smoothly, starting from exploring available halls and packages, checking availability, submitting a reservation request, and completing required payments without requiring external guidance. Similarly, administrators must be able to efficiently manage bookings, payments, and system information through an organized dashboard interface.

6.5 Scalability

The HallCamilla system must be designed with future scalability in mind, allowing the application to accommodate increased usage and additional functionality over time. Although the current implementation is developed specifically for a single hotel location, the serverless Firebase architecture provides the ability to automatically handle variations in user activity, increased data volume, and higher numbers of booking requests without requiring manual server management.

The system must utilize the scalability advantages of cloud-based infrastructure to support potential growth in customer interactions, database operations, and application usage. This ensures that performance and reliability can be maintained even during periods of increased demand.

Furthermore, the React-based application must follow a modular and maintainable code structure, allowing future developers to introduce additional features, such as new payment methods, expanded event services, or enhanced management functionalities, without requiring significant changes to the existing system architecture. This flexible design approach ensures that HallCamilla can evolve according to future business requirements.

6.6 Maintainability

The HallCamilla system must be developed with a maintainable and organized code structure to support future modifications and improvements. The source code must follow modular design principles, separating business logic, database operations, and user interface components for better readability and management.

The application must utilize TypeScript typing practices to reduce errors and improve code reliability. Proper documentation and clean coding standards must be maintained to help future developers understand the system, identify issues, and implement updates efficiently.

7. APPENDICES

7.1 Assumptions

This section describes the main assumptions considered during the design and development of the HallCamilla system. It is assumed that customers and administrators have reliable internet access, as the cloud-based architecture requires online connectivity to perform system operations.

It is also assumed that Firebase services, including authentication and Cloud Firestore, remain available to support the application's core functions. Additionally, hotel administrators are expected to regularly manage bookings, update hall and package information, and maintain accurate system data to ensure smooth operation.

7.2 Dependencies

The HallCamilla system relies on several external technologies and services for its continuous operation. The application depends on Firebase Authentication for user management, Cloud Firestore for data storage, and Firebase Hosting for deploying and delivering the web application.

The frontend interface depends on React.js and Tailwind CSS for application development and styling. In addition, payment processing relies on the PayHere Sandbox API, while automated email notifications depend on the EmailJS service. The availability of these external services is essential for maintaining the system's core functionalities.

7.3 Acronyms

Acronym	Full Form	Description
API	Application Programming Interface	Allows communication between software systems and external services.
CRUD	Create, Read, Update, Delete	Basic operations used to manage database records.
DB	Database	Stores and manages application data.
HTTPS	Hypertext Transfer Protocol Secure	Provides secure encrypted data communication.

JSON	JavaScript Object Notation	Format used for data exchange between systems.
NoSQL	Not Only SQL	Flexible document-based database model used by Firestore.
PDF	Portable Document Format	File format used for receipts and reports.
RBAC	Role-Based Access Control	Controls access based on user roles.
SPA	Single Page Application	Updates pages dynamically without full reloads.
SRS	Software Requirements Specification	Defines system requirements and specifications.
UI	User Interface	Visual elements users interact with.
UX	User Experience	Overall user interaction and satisfaction with the system.

7.4 Use Case Diagram

The Use Case Diagram illustrates the interaction between the two main system actors, the **Customer** and the **Administrator**, and the functionalities provided by the HallCamilla system. It shows customer activities such as registration, browsing halls and packages, submitting bookings, making payments, downloading receipts, and submitting reviews. It also represents administrator activities including managing bookings, halls, packages, payments, cancellations, and viewing revenue information.

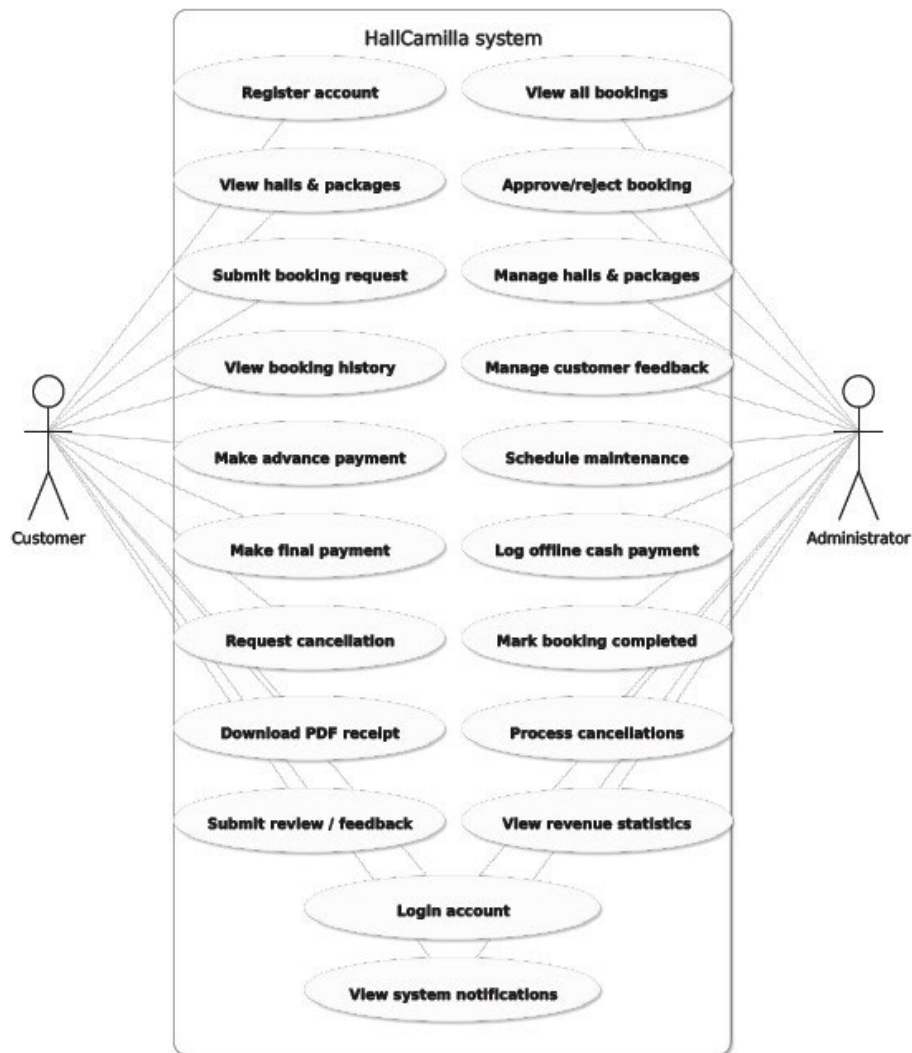


Figure 7 . 1 : Use Case Diagram

7.5 Entity Relationship Diagram

The Entity Relationship Diagram below depicts the complete logical data structure underpinning the HallCamilla system's Firestore database. It illustrates the core entities, namely User, Hall, Package, Booking, Payment, Review, and Notification, along with the relationships that connect them, such as a User making a Booking, a Booking belonging to a Package, a Hall having a Booking, and a Booking generating a Payment. This structure ensures that all operational and financial data remains logically organized and consistently related across the entire application.

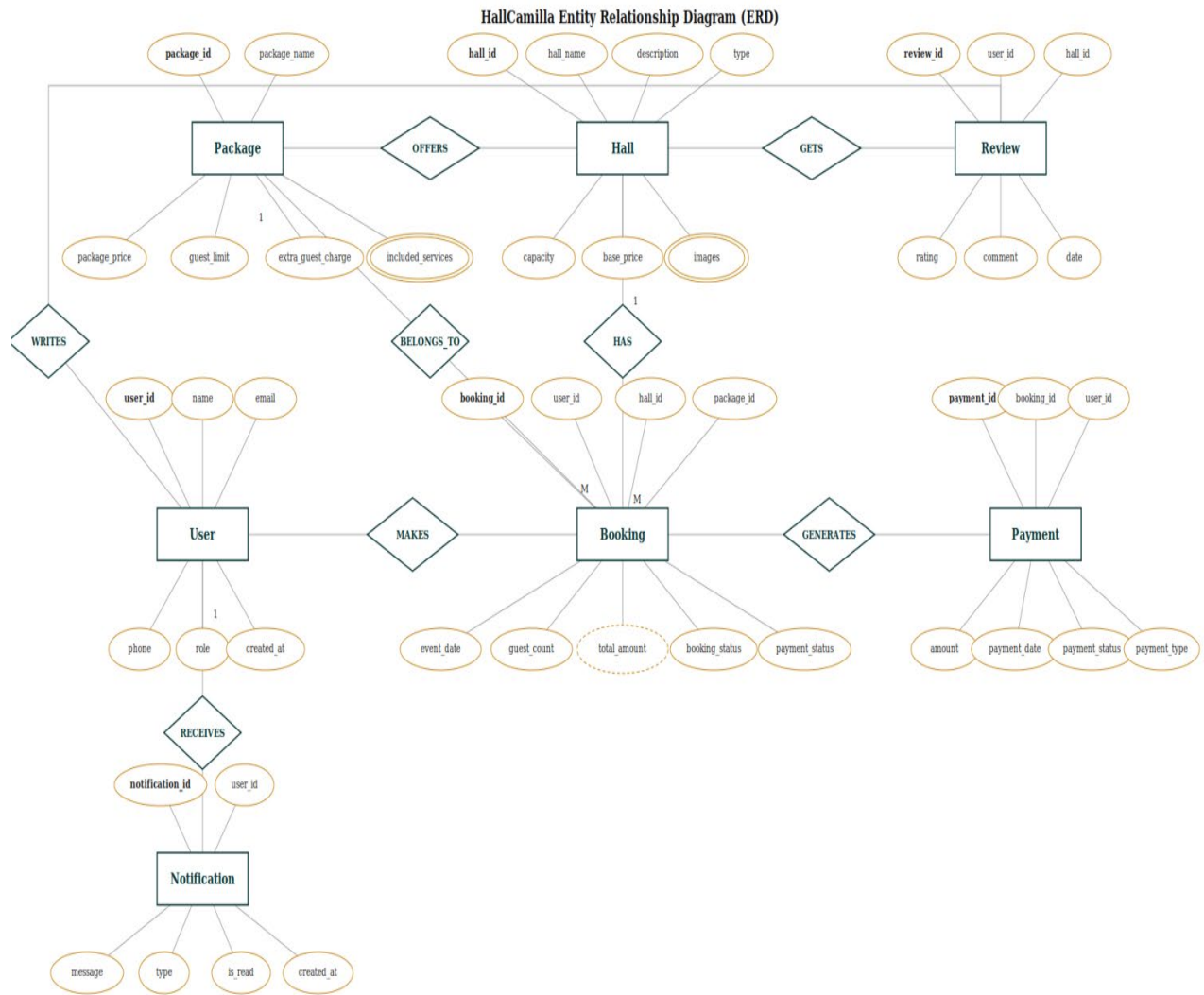


Figure 7 . 2 : Entity Relationship Diagram

7.6 References

- Firebase Documentation. Available at: <https://firebase.google.com/docs>
- React Documentation. Available at: <https://react.dev/>
- Vite Documentation. Available at: <https://vite.dev/guide/>
- Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
- TypeScript Documentation. Available at: <https://www.typescriptlang.org/docs/>
- PayHere Developer Documentation. Available at: <https://support.payhere.lk/>
- EmailJS Documentation. Available at: <https://www.emailjs.com/docs/>
- Git Documentation. Available at: <https://git-scm.com/doc>
- GitHub Documentation. Available at: <https://docs.github.com/>
- IEEE. Software Requirements Specification Standards and Guidelines.